Data Structures and Algorithms
Lab 4 – Midterm Revision + Recursion
Version 4 – Transportation and Logistics

Full tutorials, Java implementations, worked traces, and four required tasks

exercises.

Workload: four required problems, not nine

Only Problems 1–4 are mandatory. Problems 5–10 are optional enrichment. Each required problem is one VJudge submission in Java, although it may use helper classes or methods. The review/demo code in this handout is instructional and does not create extra submissions.

Section	Covered knowledge / expected output
Revision A	Bubble/insertion/selection step traces; iterative binary search (found + not found); Java code.
Revision B	Custom singly/doubly/circular lists; pointer repair and boundaries; Java code.
Revision C	Array/linked/min/monotonic stacks; safe APIs; Java code.
Revision D	Linear/circular/linked queues, deque, priority and monotonic deque; Java code.
New topic	Recursion, call-stack traces, base cases, linear recursion and divide-and-conquer; Java code.
Practice	Four required problems; six optional problems; direct Java submission in VJudge; an optional short reasoning log.

Study sequence and testing expectations

1. First read the traces, then run the five example Java demo programs. Try modifying one input and predict the output before executing.
2. For sorting, record the state after a full outer pass/iteration, not after an arbitrary inner comparison. For binary search, record every low, high, mid and comparison.
3. For linked lists, queues and stacks, draw the exact internal state after operations involving an empty or single-element structure.
4. For recursion, write the base case and the smaller subproblem, then trace both call descent and return ascent.
5. Only after completing the revision proceed to the four required problems and test each with normal, boundary and tricky cases.

Part I – Midterm Revision: Sorting and Searching

Revision 1. Sorting: what the three algorithms do

Sorting rearranges values in a chosen order. This lab revisits bubble sort, insertion sort, and selection sort from the earlier sorting laboratory. Each is implemented manually, so using Arrays.sort is not a substitute for the requested review. We use a single starting array to make their differences easy to compare: [7, 4, 8, 3, 5]. All positions in this handout are zero-based.

Method	Central idea
Bubble sort	Compare adjacent elements and swap inverted pairs; after a complete pass, the largest remaining element reaches its final place.
Insertion sort	Maintain a sorted prefix; shift larger values right and insert the next key into its correct position.
Selection sort	Select the smallest element in the remaining suffix, then put it at the first unsorted position.

Bubble sort: a complete pass-by-pass trace

At the start of a pass, compare each adjacent pair inside the still-unsorted prefix. Swapping a larger left value with a smaller right value moves the larger one toward the end. An early-exit flag allows stopping if an entire pass makes no swaps.

Stage	Array after stage	Explanation
Start	[7, 4, 8, 3, 5]	No values are known to be in their final positions.
Pass 1	[4, 7, 3, 5, 8]	Swap 7/4, 8/3, and 8/5; 8 is now in its final place.
Pass 2	[4, 3, 5, 7, 8]	Swap 7/3 and 7/5; 7 is fixed.
Pass 3	[3, 4, 5, 7, 8]	Swap 4/3; remaining suffix stays ordered.
Pass 4	[3, 4, 5, 7, 8]	No swap; early exit confirms the entire array is sorted.

Inside Pass 1: compare (7,4) -> swap; (7,8) -> keep; (8,3) -> swap; (8,5) -> swap. Do not confuse a single comparison with a whole pass.

Insertion sort: one new key at a time

The prefix ending at i-1 is sorted before iteration i. Save a[i] as key; shift every larger value to the right until the insertion position is found. Equal values need not be moved when the strict greater-than comparison is used.

i / key	Array after insertion	Reason
1 / 4	[4, 7, 8, 3, 5]	Move 7 right; put 4 at index 0.
2 / 8	[4, 7, 8, 3, 5]	8 is not smaller than the sorted prefix tail.
3 / 3	[3, 4, 7, 8, 5]	Shift 8, 7, and 4, then place 3 at index 0.
4 / 5	[3, 4, 5, 7, 8]	Shift 8 and 7, then insert 5 after 4.

Selection sort: choose the minimum remaining element

At outer iteration i, find the index of the minimum in positions i through n-1; swap this minimum with position i. The prefix through i is fixed after that iteration. Selection sort always scans the remaining suffix, even if the input is already sorted.

i / minimum	Array after selection	What becomes fixed
0 / 3	[3, 4, 8, 7, 5]	3 at index 0.
1 / 4	[3, 4, 8, 7, 5]	4 at index 1; swap with itself.
2 / 5	[3, 4, 5, 7, 8]	5 at index 2.
3 / 7	[3, 4, 5, 7, 8]	7 at index 3; 8 is already last.

Sorting complexities and when to choose them

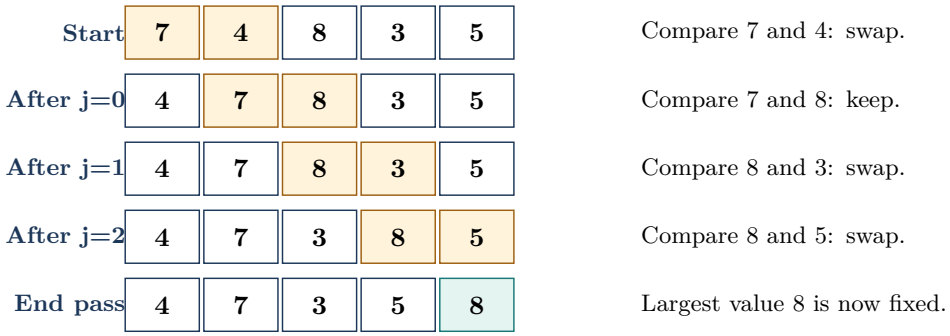
Algorithm	Best time	Worst time	Extra space / stability
Bubble (early exit)	O(n)	O(n^2)	O(1), stable with strict > comparisons
Insertion	O(n)	O(n^2)	O(1), stable with strict > comparisons
Selection	O(n^2)	O(n^2)	O(1), not generally stable

Bubble sort is instructive for adjacent swaps and inversions, insertion sort works well for small or nearly sorted inputs, and selection sort limits swaps but still makes quadratic comparisons. They are learning algorithms, not ideal choices for large general-purpose datasets.

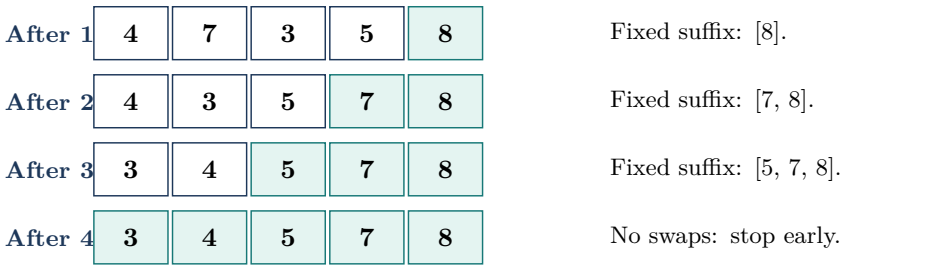
Visual A. Bubble sort: each comparison and each pass

The amber cells are the adjacent values currently compared; turquoise marks the suffix already in its final sorted position. Follow the algorithm from left to right, then continue down to the next row.

Inside Pass 1: four adjacent comparisons



Completed outer passes



Invariant: after pass p, the last p values are final. Within a pass, a larger adjacent value moves right after each swap. **Self-check:** if the input were already sorted, how many full passes would the early-exit version make?

Visual B. Insertion and selection: two different invariants

Both methods produce the same ascending order, but they move values differently. Turquoise is the sorted or permanently fixed prefix; amber marks the key or newly chosen minimum.

Insertion sort: shift values right to make a hole for the key

Initial	7	4	8	3	5	Key 4; shift 7 right.
Key 4	4	7	8	3	5	Prefix [4, 7] is sorted.
Key 8	4	7	8	3	5	No shift is needed.
Key 3	3	4	7	8	5	Shift 8, 7, 4 right; insert 3.
Key 5	3	4	5	7	8	Shift 8 and 7; insert 5.

Selection sort: locate minimum in suffix, then swap once

i=0	3	4	8	7	5	Choose min=3 (old index 3).
i=1	3	4	8	7	5	Choose min=4 (already at i).
i=2	3	4	5	7	8	Choose min=5 (old index 4).
i=3	3	4	5	7	8	Choose min=7: sorted.

Key distinction: insertion keeps a *sorted prefix* and may shift many values. Selection keeps a prefix of *final minimum values* and usually makes at most one swap per outer iteration. Insertion and strict-comparison bubble are stable; ordinary selection is not generally stable.

Complete Java implementation: three sorts and iterative binary search

Save the following complete program as SortingSearchDemo.java. Its main method verifies that the three algorithms produce identical ascending arrays. The search function is discussed next.

```
import java.util.Arrays;

public class SortingSearchDemo {
    static void bubbleSort(int[] a) {
        for (int end = a.length - 1; end > 0; end--) {
            boolean swapped = false;
            for (int j = 0; j < end; j++) {
                if (a[j] > a[j + 1]) {
                    int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                    swapped = true;
                }
            }
            if (!swapped) break;
        }
    }
    static void insertionSort(int[] a) {
        for (int i = 1; i < a.length; i++) {
            int key = a[i], j = i - 1;
            while (j >= 0 && a[j] > key) {
                a[j + 1] = a[j];
                j--;
            }
            a[j + 1] = key;
        }
    }
    static void selectionSort(int[] a) {
        for (int i = 0; i + 1 < a.length; i++) {
            int min = i;
            for (int j = i + 1; j < a.length; j++)
                if (a[j] < a[min]) min = j;
            int t = a[i]; a[i] = a[min]; a[min] = t;
        }
    }
```

```
    static int binarySearch(int[] sorted, int target) {
        int lo = 0, hi = sorted.length - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (sorted[mid] == target) return mid;
            if (sorted[mid] < target) lo = mid + 1;
            else hi = mid - 1;
        }
        return -1;
    }
    public static void main(String[] args) {
        int[] original = {7, 4, 8, 3, 5};
        int[] b = original.clone(), in = original.clone(), s = original.clone();
        bubbleSort(b); insertionSort(in); selectionSort(s);
        System.out.println("Bubble: " + Arrays.toString(b));
        System.out.println("Insertion: " + Arrays.toString(in));
        System.out.println("Selection: " + Arrays.toString(s));
        System.out.println("Index of 5: " + binarySearch(b, 5));
        System.out.println("Index of 6: " + binarySearch(b, 6));
    }
}
```

Revision 2. Iterative binary search and step-by-step tracing

Binary search locates a target in a sorted array. With inclusive boundaries low and high, examine mid = low + (high-low)/2. If a[mid] is too small, discard indexes through mid; if too large, discard indexes from mid onward. Repeat until found or low > high. The sorted-input precondition is mandatory.

Exact-match search: index, not insertion position

The implementation returns one matching zero-based index, or -1 if absent. If duplicates exist, an ordinary exact-match search does not guarantee the first occurrence. A separate optional challenge teaches the first-occurrence variant.

Successful trace: sorted array [3, 4, 5, 7, 8, 11, 14], target 11.

Step	low	high	mid	a[mid]	Decision
1	0	6	3	7	$7 < 11$: set low = 4.
2	4	6	5	11	Found: return index 5.

Unsuccessful trace on the same array, target 6:

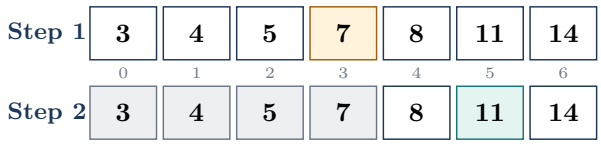
Step	low	high	mid	a[mid]	Decision
1	0	6	3	7	$7 > 6$: high = 2.
2	0	2	1	4	$4 < 6$: low = 2.
3	2	2	2	5	$5 < 6$: low = 3.
Stop	3	2	–	–	low > high: return -1.

Each step removes roughly half of the remaining candidates, so iterative binary search takes O(log n) comparisons and O(1) extra space. Searching for an element in an ordinary linked list does not offer the same random-access efficiency because reaching its midpoint requires traversal.

Visual C. Binary search: watch the search interval shrink

Index positions are written below each cell. Grey cells have been rejected, amber identifies mid, and turquoise identifies the discovered target. Every branch must remove mid from the next candidate interval unless it is the target.

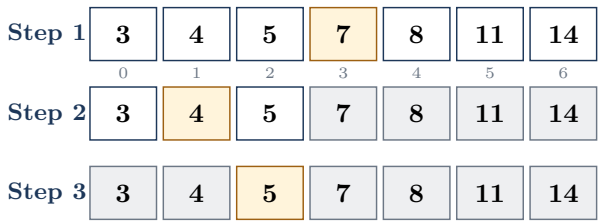
Found trace: target = 11



$a[3] = 7 < 11$, keep indices 4–6.

$a[5] = 11$, return index 5.

Not-found trace: target = 6



$a[3] = 7 > 6$; high becomes 2.

$a[1] = 4 < 6$; low becomes 2.

$a[2] = 5 < 6$; low becomes 3.

Stop condition: low = 3 and high = 2, so low > high; return −1. A failed search is not a request for the insertion position. The input must be sorted *before* the search, and a matching index among duplicates need not be the first.

From picture to Java code

`mid = low + (high - low)/2;` selects the middle retained index. After `a[mid] < target`, use `low=mid+1`; after `a[mid] > target`, use `high=mid-1`. `while (low <= high)` supports the singleton interval. The iterative algorithm uses $O(1)$ auxiliary space.

Java search snippet to read independently

```
static int binarySearch(int[] a, int target) {
    int low = 0, high = a.length - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (a[mid] == target) return mid;
        if (a[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

Checking the wrong half, updating low/high without excluding mid, or applying binary search to unsorted values are common causes of infinite loops or wrong answers.

Part II – Midterm Revision: Linked Lists

Revision 3. Nodes, references, and link invariants

A linked list stores values in nodes connected by references. Unlike an array, an ordinary linked list does not support O(1) access to its i-th element. The head locates the first node; a tail reference permits O(1) appends when maintained correctly. Removing a node means repairing references—not shifting array positions.



Variant	Links / invariant	Typical strengths
Singly	Each node points next; last.next == null.	Simple forward traversal; O(1) prepend and tail append.
Doubly	Each node has prev and next; head.prev and tail.next are null.	Forward/backward steps and O(1) deletion of a known node.
Circular singly	For a nonempty list, tail.next is head.	Repeated round-robin traversal; O(1) rotation with tail reference.
Circular doubly	For a nonempty list, head.prev is tail and tail.next is head.	Two-way cyclic traversal and known-node removal.

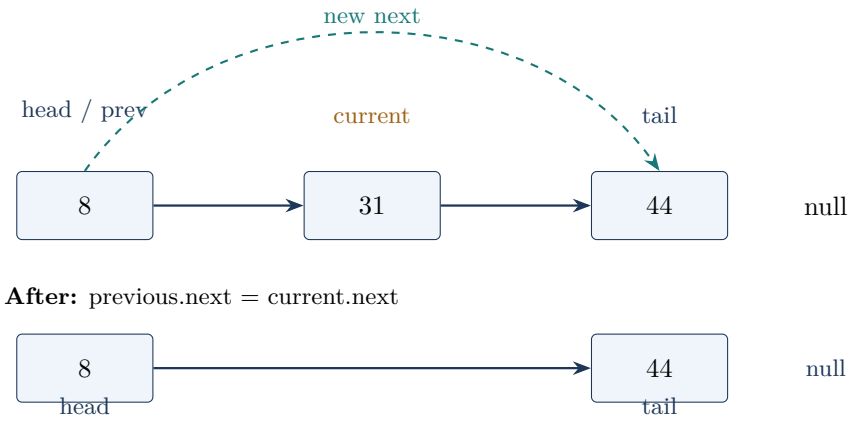
Pointer-update walkthroughs

- Singly addFirst: create node n; set n.next = head; then head = n. If previously empty, also set tail = n.
- Singly remove first matching x: keep previous and current; bypass current with previous.next = current.next. Update head or tail when the removed node is an endpoint.
- Singly reverse: repeatedly redirect current.next toward previous. After the loop, old head becomes tail and previous becomes new head.
- Doubly delete a known node: join node.prev.next to node.next and node.next.prev to node.prev, updating head/tail where a neighbor is null.
- Circular singly with tail: tail.next is the current front. To rotate, simply advance tail = tail.next; never traverse with p != null on a circular list.
- Circular doubly: adding at the rear updates four links: new.prev, new.next, oldTail.next, and head.prev.

Visual D. Linked-list links and boundary repairs

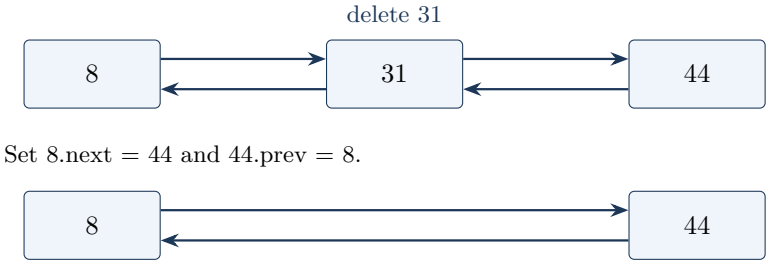
Unlike arrays, linked lists modify references. Trace which reference is redirected and which node becomes unreachable. A removed node is not deleted by shifting later values.

Singly linked list: remove the middle node 31



If the removed node is the head, set head to head.next. **If it is the tail**, set tail to previous. **If it is the only node**, both head and tail become null. Keep a reference to the next node before redirecting links when reversing a list.

Doubly linked list: both directions must remain connected

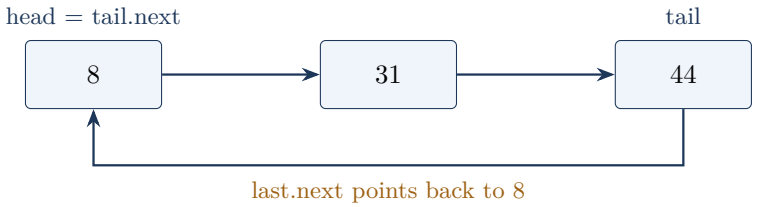


Invariant: every forward link between adjacent nodes must agree with its backward link. Deleting a *known* middle node is $O(1)$; finding the node by value is still $O(n)$.

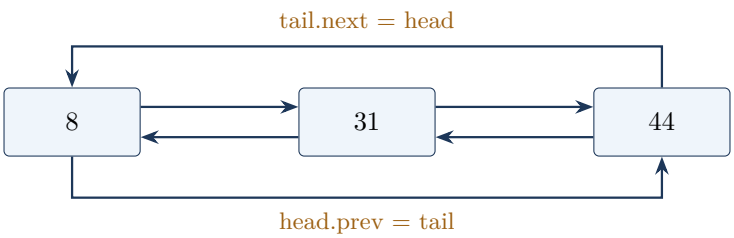
Visual E. Circular linked lists: there is no null endpoint

A circular singly linked list connects tail.next to head. A circular doubly linked list additionally connects head.prev to tail. Stopping after one complete lap is essential.

Circular singly: the tail reference identifies the whole cycle



Circular doubly: wrap in both directions



Singleton case: when one node exists, its next (and, for a doubly circular list, prev) points to itself. The condition `node != null` will *never* stop a traversal of a nonempty circular list. Instead stop when the reference returns to the start, or after a known number of nodes.

Complete Java reference: four linked-list variants

This runnable demonstration implements singly, doubly, circular singly, and circular doubly lists as separate nested classes. Note the singleton cases and finite one-lap traversal conditions. It is provided as review code, not a solution to the assigned command interfaces.

```
public class LinkedListVariantsDemo {
    // Singly: head and tail. Supports forward traversal and O(1) append.
    static class Singly {
        static class Node { int x; Node next; Node(int x) { this.x = x; } }
        Node head, tail;
        void addFirst(int x) {
            Node n = new Node(x); n.next = head; head = n;
            if (tail == null) tail = n;
        }
        void addLast(int x) {
            Node n = new Node(x);
            if (tail == null) head = tail = n;
            else { tail.next = n; tail = n; }
        }
        boolean removeFirstMatch(int x) {
            Node prev = null, cur = head;
            while (cur != null && cur.x != x) {
                prev = cur; cur = cur.next;
            }
            if (cur == null) return false;
            if (prev == null) head = cur.next;
            else prev.next = cur.next;
            if (tail == cur) tail = prev;
            return true;
        }
        String forward() {
            StringBuilder s = new StringBuilder();
            for (Node p = head; p != null; p = p.next) {
                if (s.length() > 0) s.append(' ');
                s.append(p.x);
            }
            return s.toString();
        }
    }
    // Doubly: deleting a known node repairs both links.
    static class Doubly {
        static class Node {
            int x; Node prev, next;
            Node(int x) { this.x = x; }
        }
        Node head, tail;
        void addLast(int x) {
            Node n = new Node(x); n.prev = tail;
            if (tail == null) head = n;
            else tail.next = n;
            tail = n;
        }
        void remove(Node n) { // n must belong to this list
            if (n.prev == null) head = n.next;
            else n.prev.next = n.next;
            if (n.next == null) tail = n.prev;
            else n.next.prev = n.prev;
        }
        String backward() {
            StringBuilder s = new StringBuilder();
            for (Node p = tail; p != null; p = p.prev) {
                if (s.length() > 0) s.append(' ');
                s.append(p.x);
            }
            return s.toString();
        }
    }
    // Circular singly: tail.next is head for a nonempty list.
    static class CircularSingly {
        static class Node { int x; Node next; Node(int x) { this.x = x; } }
        Node tail;
```

```
        void addLast(int x) {
            Node n = new Node(x);
            if (tail == null) { tail = n; n.next = n; }
            else { n.next = tail.next; tail.next = n; tail = n; }
        }
        int takeFront() {
            if (tail == null) throw new IllegalStateException("EMPTY");
            Node first = tail.next;
            if (first == tail) tail = null;
            else tail.next = first.next;
            return first.x;
        }
        String oneLap() {
            if (tail == null) return "";
            StringBuilder s = new StringBuilder();
            Node p = tail.next;
            do {
                if (s.length() > 0) s.append(' ');
                s.append(p.x);
                p = p.next;
            } while (p != tail.next);
            return s.toString();
        }
    }
    // Circular doubly: head.prev = tail and tail.next = head.
    static class CircularDoubly {
        static class Node {
            int x; Node prev, next;
            Node(int x) { this.x = x; }
        }
        Node head;
        void addLast(int x) {
            Node n = new Node(x);
            if (head == null) { head = n; n.next = n.prev = n; }
            else {
                Node tail = head.prev;
                n.prev = tail; n.next = head;
                tail.next = n; head.prev = n;
            }
        }
        void remove(Node n) { // n must belong to this list
            if (n.next == n) { head = null; return; }
            n.prev.next = n.next;
            n.next.prev = n.prev;
            if (head == n) head = n.next;
        }
        String forwardOneLap() {
            if (head == null) return "";
            StringBuilder s = new StringBuilder();
            Node p = head;
            do {
                if (s.length() > 0) s.append(' ');
                s.append(p.x);
                p = p.next;
            } while (p != head);
            return s.toString();
        }
    }
    public static void main(String[] args) {
        Singly a = new Singly();
        a.addLast(4); a.addLast(8); a.addFirst(2); a.removeFirstMatch(4);
        System.out.println("Singly: " + a.forward());
        Doubly b = new Doubly();
        b.addLast(3); b.addLast(7); b.addLast(9); b.remove(b.head.next);
        System.out.println("Doubly backward: " + b.backward());
        CircularSingly c = new CircularSingly();
        c.addLast(10); c.addLast(20); c.addLast(30);
        System.out.println("Circular singly: " + c.oneLap());
        System.out.println("Taken: " + c.takeFront());
        CircularDoubly d = new CircularDoubly();
        d.addLast(1); d.addLast(2); d.addLast(3); d.remove(d.head.prev);
        System.out.println("Circular doubly: " + d.forwardOneLap());
    }
}
```

Costs and important boundary conditions

Prepending a singly list and appending with a maintained tail take O(1). Searching by value and removing by searched value take O(n). In a doubly list, removing an already known node takes O(1), but finding it first can still take O(n). Empty and singleton updates are especially important: after deleting the final node, both head and tail must represent an empty list. A circular-list traversal must have an explicit stopping condition.

Part III – Midterm Revision: Stacks

Revision 4. Stack ADT and its implementations

A stack is a Last-In, First-Out (LIFO) abstract data type. Push adds a top element; pop removes that element; peek reads without removing it. The operations define behavior, while the choice of an array or linked nodes determines implementation details.

Operation	Stack bottom to top	Result
push(3)	[3]	–
push(8)	[3,8]	–
push(2)	[3,8,2]	–
peek()	[3,8,2]	2
pop()	[3,8]	2

Variant	Implementation idea	Typical cost
Growing array stack	Top at a[size-1]; double storage when full.	Push amortized O(1); pop/peek O(1).
Linked stack	Head node is the top; push/pop modify head.	O(1) push/pop/peek, extra node references.
Min stack	Use a second stack recording the minimum at each depth.	O(1) push/pop/getMin with extra O(n) memory.
Monotonic stack	Retain unresolved indices with ordered candidate values.	O(n) for next-greater/next-smaller scans, O(n) space.

Java ArrayDeque supports push/pop/peek for most application stack tasks. Use a manual array or node stack when the assignment explicitly asks you to implement the data structure. Do not confuse a monotonic stack with sorting the entire input: original index order is essential.

Worked monotonic-stack trace: next greater element

For values [4,2,6,3], scan from left to right. The stack stores indices whose next strictly greater value is still unknown. When a larger value arrives, resolve and pop each smaller unresolved value.

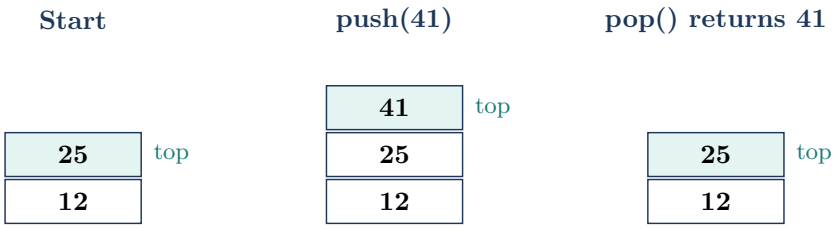
Read i / value	Unresolved indices after step	Resolved new answers
0 / 4	[0]	None
1 / 2	[0,1]	None
2 / 6	[2]	index 1 -> 6; index 0 -> 6
3 / 3	[2,3]	None
End	–	indexes 2 and 3 -> -1

The answer is [6, 6, -1, -1]. Each index is pushed once and popped at most once, yielding O(n) total work even though a while-loop appears inside the for-loop.

Visual F. Stacks: LIFO, min stack, and monotonic stack

The top is the only accessible removal point. Compare physical storage with the logical stack order; a linked stack and an array stack implement the same LIFO behavior.

Array/linked stack: push and pop change only the top



Min stack: store one running minimum for each stack level

Operation	Data stack (bottom to top)	Min stack (bottom to top)
push 7	[7]	[7]
push 3	[7, 3]	[7, 3]
push 5	[7, 3, 5]	[7, 3, 3]
push 2	[7, 3, 5, 2]	[7, 3, 3, 2]
pop 2	[7, 3, 5]	[7, 3, 3]

If the current minimum is tracked in a second stack, pushing x also pushes $\min(x, \text{currentMin})$. Both levels must be popped together. For all-negative and duplicate inputs, the rule still works.

Monotonic stack: unresolved indices for next strictly greater element

Input step	Pending indices	Event
a[0]=4	[0]	0 waits for a larger value.
a[1]=2	[0,1]	1 also waits.
a[2]=5	[2]	5 resolves 1 and 0.
a[3]=3	[2,3]	No pending index resolved.
Finish	[2,3]	Both have no greater future value.

Result: for $[4, 2, 5, 3]$, next-greater *values* are $[5, 5, -1, -1]$. Every index is pushed once and popped at most once, explaining the overall $O(n)$ time. This is a specialized pattern, not a rule for ordinary LIFO operations.

Complete Java reference: array, linked, min, and monotonic stacks

```
import java.util.ArrayDeque;
import java.util.Arrays;
import java.util.Deque;
import java.util.NoSuchElementException;

public class StackVariantsDemo {
    // Array stack: doubling makes push amortized O(1).
    static class ArrayStack {
        int[] a = new int[2]; int size;
        void push(int x) {
            if (size == a.length) a = Arrays.copyOf(a, size * 2);
            a[size++] = x;
        }
        int pop() {
            if (size == 0) throw new NoSuchElementException();
            return a[--size];
        }
        int peek() {
            if (size == 0) throw new NoSuchElementException();
            return a[size - 1];
```

```
        }
    }
    // Linked stack: head is top; no shifting or resizing.
    static class LinkedStack {
        static class Node { int x; Node next; Node(int x, Node n) { this.x=x; next=n; } }
        Node head;
        void push(int x) { head = new Node(x, head); }
        int pop() {
            if (head == null) throw new NoSuchElementException();
            int x = head.x; head = head.next; return x;
        }
    }
    // Min stack: second stack remembers minimum at each depth.
    static class MinStack {
        Deque<Integer> data = new ArrayDeque<>();
        Deque<Integer> mins = new ArrayDeque<>();
        void push(int x) {
            data.push(x);
            mins.push(mins.isEmpty() ? x : Math.min(x, mins.peek()));
        }
        int pop() {
            mins.pop(); return data.pop();
        }
        int min() {
            if (mins.isEmpty()) throw new NoSuchElementException();
            return mins.peek();
        }
    }
    // Monotonic stack: next strictly greater value to the right.
    static int[] nextGreater(int[] a) {
        int[] result = new int[a.length];
        Arrays.fill(result, -1);
        Deque<Integer> unresolved = new ArrayDeque<>(); // indices
        for (int i = 0; i < a.length; i++) {
            while (!unresolved.isEmpty() && a[unresolved.peek()] < a[i]) {
                result[unresolved.pop()] = a[i];
            }
            unresolved.push(i);
        }
        return result;
    }
    public static void main(String[] args) {
        ArrayStack a = new ArrayStack(); a.push(3); a.push(5);
        System.out.println("Array pop: " + a.pop());
        LinkedStack b = new LinkedStack(); b.push(12); b.push(15);
        System.out.println("Linked pop: " + b.pop());
        MinStack c = new MinStack(); c.push(9); c.push(2); c.push(7);
        System.out.println("Minimum: " + c.min());
        c.pop(); c.pop(); System.out.println("Minimum after pop: " + c.min());
        System.out.println("Next greater: " + Arrays.toString(nextGreater(new int[]{4,2,6,3})));
    }
}
```

Part IV – Midterm Revision: Queues and Variants

Revision 5. Queue, circular queue, and linked queue

A queue is First-In, First-Out (FIFO). Enqueue adds to the back, dequeue removes from the front, and peek reads the oldest item. Repeated removal from the beginning of an ordinary array may require shifting all remaining items. A circular queue instead moves an index: rearPosition = (front + size) % capacity.

Step	front	size	Logical queue
start	0	0	[]
enqueue(14)	0	1	[14]
enqueue(27)	0	2	[14,27]
enqueue(35)	0	3	[14,27,35]
dequeue()	1	2	[27,35]
enqueue(46)	1	3	[27,35,46] (46 stored at index 0)

The key invariant is that the logical i-th element is stored at a[(front+i) % capacity], for 0 <= i < size. A linked queue instead keeps head and tail and allocates one node per arrival. When removing the last linked node, set tail = null as well as head = null.

Deque, priority queue, and monotonic deque

Variant	Rule and sample use	Typical cost
Linear array queue	Shift elements after dequeue; simple but inefficient.	Enqueue O(1), dequeue O(n).
Circular array queue	Fixed ring buffer reuses slots via modulo.	O(1) enqueue/dequeue/peek.
Linked queue	Append at tail, remove head.	O(1) enqueue/dequeue/peek.
Deque	Both ends support insertion/removal (ArrayDeque).	Amortized O(1) end operations.
Priority queue	Serve lowest numeric priority first; tie order needs explicit comparator.	Heap offer/poll O(log n), peek O(1).
Monotonic deque	Maintain candidates for max/min of each sliding window.	O(n) across n values; deque space O(k).

Worked monotonic-deque trace: window maxima

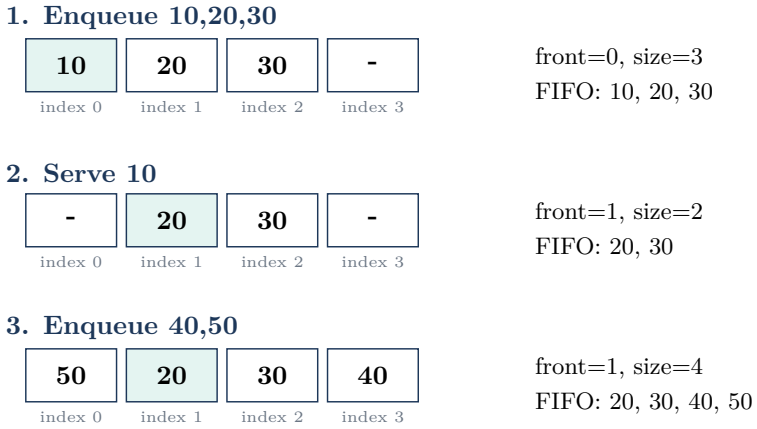
For a = [3,8,2,6,4,9,1], k = 3, remove expired indices at the front and dominated smaller-or-equal values from the rear. Store indices, not just values, so each candidate can be expired when its window ends.

Window	Values	Maximum
0..2	[3,8,2]	8
1..3	[8,2,6]	8
2..4	[2,6,4]	6
3..5	[6,4,9]	9
4..6	[4,9,1]	9

The maxima are [8,8,6,9,9]. The monotonic deque keeps only candidates that might matter for a future complete window; the input array itself is not rearranged.

Visual G. Circular queues: physical slots versus FIFO order

A ring buffer keeps front and size. Enqueue writes at (front+size) modulo capacity; dequeue advances front modulo capacity. The physical array can look unsorted while the logical order remains FIFO.



Wrap-around: after step 2, position 0 is unused but the queue is not empty. At step 3, 40 uses index 3 and 50 wraps to index 0. A filled array of four elements is logically $[20, 30, 40, 50]$, not physical order $[50, 20, 30, 40]$. Failed enqueue on full and failed dequeue on empty must leave front/size unchanged.

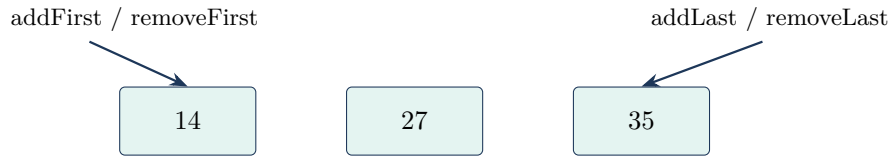
Compare implementation invariants

Linear array queue: serving with shifts costs $O(n)$. **Circular array:** no shifting; the ith live item is `a[(front+i)%K]`. **Linked queue:** head is the oldest item and tail the newest; removing the only node resets both to null.

Visual H. Deque, priority queue, and monotonic deque

These structures are not interchangeable. Use the operation and ordering rule to select the correct variant for a problem.

Deque: both ends are accessible



Priority queue: urgency is the ordering key

Arrival	Case ID	Priority (lower first)
0	101	3
1	102	1
2	103	1

If the comparator is $(\text{priority}, \text{arrivalIndex})$, the removal order is $102 \to 103 \to 101$. Without the secondary key, a Java priority queue is not guaranteed to maintain FIFO order for equal-priority items.

Monotonic deque for sliding-window maximum, $k = 3$

Step	Deque (index:value)	Update and reported answer
i=0	[0:3]	No complete window
i=1	[1:8]	3 is dominated; remove from rear
i=2	[1:8, 2:2]	Window 0–2: max 8
i=3	[1:8, 3:6]	Window 1–3: max 8
i=4	[3:6, 4:4]	Index 1 expires; max 6
i=5	[5:9]	9 removes smaller candidates; max 9
i=6	[5:9, 6:1]	Window 4–6: max 9

Every index enters once and leaves at most once. Store *indices* to detect expiration and use their array values for dominance comparisons. The output is $[8, 8, 6, 9, 9]$, and the input is never re-sorted.

Complete Java reference: all six queue-family variants

This runnable demonstration includes a simple linear-array queue, circular-array queue, linked queue, deque, priority queue with FIFO ties, and monotonic deque. The ordinary Java Queue interface is also discussed through these implementations.

```
import java.util.*;

public class QueueVariantsDemo {
    // Simple array queue: shifts after removal; O(n) dequeue.
    static class LinearQueue {
        final int[] a; int size;
        LinearQueue(int cap) { a = new int[cap]; }
        boolean offer(int x) {
            if (size == a.length) return false;
            a[size++] = x; return true;
        }
        int poll() {
            if (size == 0) throw new NoSuchElementException();
            int value = a[0];
            for (int i = 1; i < size; i++) a[i-1] = a[i];
            size--; return value;
        }
    }
    // Circular array queue: no shifts; O(1) per operation.
    static class CircularQueue {
        final int[] a; int front, size;
        CircularQueue(int cap) {
```

```
            if (cap < 1) throw new IllegalArgumentException();
            a = new int[cap];
        }
        boolean offer(int x) {
            if (size == a.length) return false;
            a[(front + size) % a.length] = x;
            size++; return true;
        }
        int poll() {
            if (size == 0) throw new NoSuchElementException();
            int x = a[front]; front = (front + 1) % a.length;
            size--; return x;
        }
    }
    // Linked queue: maintain both front/head and rear/tail.
    static class LinkedQueue {
        static class Node { int x; Node next; Node(int x) { this.x=x; } }
        Node head, tail;
        void offer(int x) {
            Node n = new Node(x);
            if (tail == null) head = tail = n;
            else { tail.next = n; tail = n; }
        }
        int poll() {
            if (head == null) throw new NoSuchElementException();
            int x = head.x; head = head.next;
            if (head == null) tail = null;
            return x;
        }
    }
    // Deque: add/remove at both ends.
    static void demoDeque() {
        Deque<Integer> d = new ArrayDeque<>();
        d.addFirst(7); d.addLast(8); d.addFirst(6);
        System.out.println("Deque ends: " + d.removeFirst() + "," + d.removeLast());
    }
    // PriorityQueue: urgent small number first, FIFO on ties.
    static class Job {
        int id, priority, arrival;
        Job(int id, int priority, int arrival) {
            this.id=id; this.priority=priority; this.arrival=arrival;
        }
    }
    static void demoPriority() {
        PriorityQueue<Job> pq = new PriorityQueue<>(
            Comparator.comparingInt((Job j) -> j.priority)
                      .thenComparingInt(j -> j.arrival));
        pq.add(new Job(41,2,0)); pq.add(new Job(42,1,1));
        pq.add(new Job(43,1,2));
        System.out.println("Priority order: " + pq.poll().id + " "
            + pq.poll().id + " " + pq.poll().id);
    }
    // Monotonic deque: O(n) maxima of windows of width k.
    static int[] windowMax(int[] a, int k) {
        if (k < 1 || k > a.length) throw new IllegalArgumentException();
        int[] ans = new int[a.length - k + 1];
        Deque<Integer> q = new ArrayDeque<>(); // candidate indices
        for (int i = 0; i < a.length; i++) {
            while (!q.isEmpty() && q.peekFirst() <= i - k) q.removeFirst();
            while (!q.isEmpty() && a[q.peekLast()] <= a[i]) q.removeLast();
            q.addLast(i);
            if (i >= k-1) ans[i-k+1] = a[q.peekFirst()];
        }
        return ans;
    }
    public static void main(String[] args) {
        LinearQueue l = new LinearQueue(2);
        l.offer(4); l.offer(6); System.out.println("Linear dequeue: " + l.poll());
        CircularQueue c = new CircularQueue(2);
        c.offer(10); c.offer(20); c.poll(); c.offer(30);
        System.out.println("Circular dequeue: " + c.poll());
        LinkedQueue q = new LinkedQueue(); q.offer(9); q.offer(11);
        System.out.println("Linked dequeue: " + q.poll());
        demoDeque(); demoPriority();
        System.out.println("Window max: " + Arrays.toString(
            windowMax(new int[]{3,8,2,6,4,9,1}, 3)));
    }
}
```

Part V – New Topic: Recursion

Recursion: base case, recursive case, and progress

Recursion is a technique in which a method solves a problem by calling itself on a smaller subproblem. Every correct terminating recursive design needs (1) a base case that returns immediately, (2) a recursive case that reduces the problem, and (3) progress toward the base case. Without these, calls may continue until a StackOverflowError.

Think in contracts, not just calls

State what one call promises to compute. For example, sumArray(a,i) returns the sum of elements from index i to the end. At i == a.length, that suffix is empty and its sum is zero. Otherwise the answer is a[i] plus the recursively computed remaining suffix.

Full call-stack trace: factorial(4)

Use factorial(n) = n * factorial(n-1) for n>1, and factorial(0)=factorial(1)=1. Java must finish the deepest call before returning values outward.

Call descent	Return ascent
factorial(4) waits for factorial(3)	factorial(1) returns 1
factorial(3) waits for factorial(2)	factorial(2) returns 2 * 1 = 2
factorial(2) waits for factorial(1)	factorial(3) returns 3 * 2 = 6
factorial(1) is a base case	factorial(4) returns 4 * 6 = 24

This requires O(n) recursive calls and O(n) call-stack space. For the sample Java long implementation, factorial input is restricted to 0..20 to avoid overflow; 21! does not fit in a signed long.

Different recursive patterns

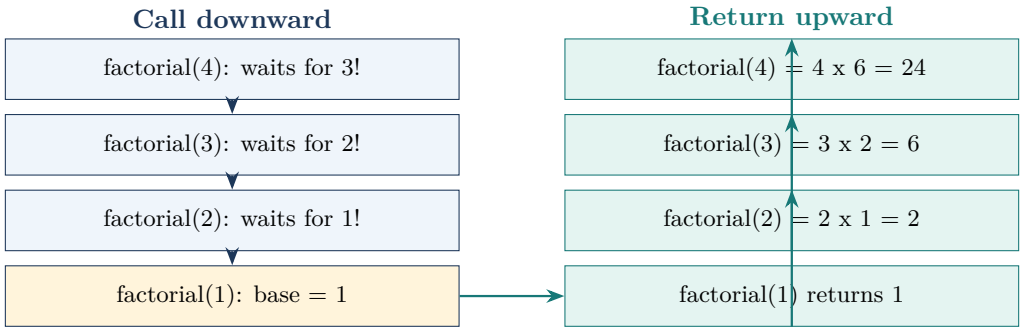
Pattern	Base case	Reduction
Single decreasing integer	n <= 1	factorial(n-1)
Array suffix	i == a.length	sumArray(a,i+1)
Two-pointer recursion	left >= right	palindrome(left+1,right-1)
Recursive binary search	low > high	Recurse on one half of sorted array
Divide and conquer	One-element interval	Solve both halves and combine their answers

Input can still be read with an ordinary loop: the recursion requirement concerns the computation. Avoid changing an argument without making it smaller. Compared with iteration, recursion often uses additional call-stack space; divide-and-conquer recursion can use O(log n) depth while visiting O(n) elements.

Visual I. Recursion: stack frames descend and returns unwind

Every recursive call gets its own parameter values and suspended calculation. The recursion must make progress toward a base case; then results return in reverse order.

Call descent and return ascent for factorial(4)



Important: the *call stack* is not the same as the *stack ADT assignment*. Java automatically allocates each call frame, including its parameters and return address. This example uses $O(n)$ call depth and $O(n)$ stack space. Factorial overflows a 64-bit signed `long` after $n = 20$.

A second example: recursive suffix sum of [2, 4, 6]

Call	Returned result
sum(a,3)	0 (empty suffix: base)
sum(a,2)	6 + 0 = 6
sum(a,1)	4 + 6 = 10
sum(a,0)	2 + 10 = 12

Self-check: for each new recursive method, write its contract, base case, smaller argument, and return combination *before* writing Java code. Recursive binary search uses $O(\log n)$ call-stack space; the iterative version uses $O(1)$.

Complete Java reference: five recursive methods

```
import java.util.Arrays;

public class RecursionDemo {
    static long factorial(int n) {
        if (n < 0 || n > 20) throw new IllegalArgumentException("0..20 only");
        if (n <= 1) return 1;
        return n * factorial(n - 1);
    }
    static long sumArray(int[] a, int i) {
        if (i == a.length) return 0;
        return a[i] + sumArray(a, i + 1);
    }
    static int digitSum(int n) {
        if (n < 10) return n;
        return (n % 10) + digitSum(n / 10);
    }
    static boolean isPalindrome(int[] a, int lo, int hi) {
        if (lo >= hi) return true;
        if (a[lo] != a[hi]) return false;
        return isPalindrome(a, lo + 1, hi - 1);
    }
    static int binarySearchRecursive(int[] sorted, int lo, int hi, int t) {
        if (lo > hi) return -1;
        int mid = lo + (hi - lo) / 2;
        if (sorted[mid] == t) return mid;
        if (sorted[mid] < t)
            return binarySearchRecursive(sorted, mid+1, hi, t);
        return binarySearchRecursive(sorted, lo, mid-1, t);
    }
    public static void main(String[] args) {
        int[] a = {3, 4, 5, 7, 8};
        System.out.println("4! = " + factorial(4));
        System.out.println("Sum = " + sumArray(a, 0));
```

```
        System.out.println("Digit sum(2705) = " + digitSum(2705));
        System.out.println("Palindrome = " + isPalindrome(
            new int[]{4,2,7,2,4}, 0, 4));
        System.out.println("Index(7) = " + binarySearchRecursive(a, 0, a.length-1, 7));
    }
}
```

Self-check before the assignment

- Can you reproduce the four sorting passes and state which section of the array is already fixed?
- Can you trace the low/mid/high changes of binary search for both found and missing targets?
- Can you explain why circular queue storage positions differ from logical queue order?
- Can you repair head/tail (and prev/next) links after deleting the sole node?
- Can you recognize which tasks need LIFO, FIFO, a deque, priority ordering, or monotonic candidates?
- Can you identify the base case and the shrinking subproblem of a recursive function?

Part VI – Version 4 VJudge Problem-Solving Assignment

VJudge access and workload

Your instructor will post the correct VJudge contest link and any access code on Blackboard. The contest for your assigned version contains ten problems: A–D are **required** and E–J are **optional**. Submit in **Java**. All standard input/output comes from the online judge; use no prompts or menus. The judge statement controls the exact input, output, and constraints.

Reminder. Java method choices cannot be enforced automatically by an output-only judge. Follow the implementation technique given below and keep your accepted solution source available for instructor review. No screenshot is a substitute for VJudge acceptance.

Required problem map

Slot	Official problem	Core target
A	Selection Sort	Sorting / search
B	pushpush	Stack / queue
C	LR insertion	Linked-list links
D	Train Ticket	Recursion

How to practice each required problem

1. Read the full official problem and restate its required input, output, and limits.
2. Use the first official example for a manual trace before writing Java.
3. Choose the structure or recurrence and state its invariant or base case.
4. Implement and test normal, boundary, and tricky cases locally.
5. Submit to VJudge; read the feedback and make targeted corrections until accepted.

Required A – Selection Sort

Judge ID: `Aizu-ALDS1_2_B` **Estimated level:** Easy
Official task: Open the problem on VJudge
Revision focus: Selection sort; swap tracing.

Understand the problem

Implement selection sort and report the sorted values and number of swaps.

Suggested implementation (read after making your own plan)

At each position, locate a smallest item in the remaining suffix; swap only when necessary.

Trace before submitting

Trace the chosen minimum index and resulting array after each outer iteration.

Test cases to design

already sorted; many equal elements; descending input
Complexity target: O(n^2) time in all cases; O(1) extra

Your deliverable

Submit one accepted Java solution directly in the assigned VJudge contest. Keep the code readable enough to show the intended technique. The complete statement, input limits, and exact judge output format are defined on the linked VJudge page.

Required B – pushpush

Judge ID: `AtCoder-arc077_a` **Estimated level:** Easy–Medium
Official task: Open the problem on VJudge
Revision focus: Deque; reversing sequence logic.

Understand the problem

Repeatedly append a value and reverse the entire sequence; output the final order.

Suggested implementation (read after making your own plan)

Represent the changing orientation with a deque or reason backward about final positions; never reverse all n elements each time.

Trace before submitting

Simulate the first four operations; what is the output for odd versus even n?

Test cases to design

n=1; n=2; repeated values
Complexity target: O(n) time with deque/orientation; O(n) storage

Your deliverable

Submit one accepted Java solution directly in the assigned VJudge contest. Keep the code readable enough to show the intended technique. The complete statement, input limits, and exact judge output format are defined on the linked VJudge page.

Required C – LR insertion

Judge ID: `AtCoder-abc237_d` **Estimated level:** Medium
Official task: Open the problem on VJudge
Revision focus: Custom doubly linked list; local insertion.

Understand the problem

Insert each new integer immediately left or right of the current reference position.

Suggested implementation (read after making your own plan)

Implement Node(prev,next) links or a deque-based reverse construction; use manual nodes for the lab learning target.

Trace before submitting

Show the neighbor references changed by left insertion, then right insertion.

Test cases to design

only L; only R; alternating instructions
Complexity target: O(n) total with linked insertion; O(n) storage

Your deliverable

Submit one accepted Java solution directly in the assigned VJudge contest. Keep the code readable enough to show the intended technique. The complete statement, input limits, and exact judge output format are defined on the linked VJudge page.

Required D – Train Ticket

Judge ID: `AtCoder-abc079_c` **Estimated level:** Easy–Medium
Official task: Open the problem on VJudge
Revision focus: Recursion; plus/minus choice enumeration.

Understand the problem

Choose plus or minus between four digits so the arithmetic expression equals seven.

Suggested implementation (read after making your own plan)

Write a recursive generator choosing + or - at the three decision positions; stop when a valid expression is found.

Trace before submitting

Draw the at-most-eight leaf cases of the recursion tree. How does the base case check the total?

Test cases to design

leading zero; multiple valid formulas
Complexity target: O(2^3) constant here; O(3) recursion depth

Your deliverable

Submit one accepted Java solution directly in the assigned VJudge contest. Keep the code readable enough to show the intended technique. The complete statement, input limits, and exact judge output format are defined on the linked VJudge page.

Optional VJudge Practice – E through J

The six tasks below are optional. They are available in the same version-specific VJudge contest but are not extra mandatory Java programs. Some are stretch challenges and require more careful time-complexity reasoning. Try them after the four required problems.

Optional E – Alchemist

Judge ID: `AtCoder-abc138_c` **Estimated level:** Easy–Medium
Official task: Open the problem on VJudge
Revision focus: Sorting; greedy combination. Combine values repeatedly by averaging in an order that maximizes the final value. **Suggested approach:** Sort ascending and combine progressively, using double for fractional values. **Practice check:** Compare combining the two smallest first versus combining the largest first. **Complexity guide:** O(n log n) to sort; O(1) working after sort

Optional F – Areas on the Cross-Section Diagram

Judge ID: `Aizu-ALDS1_3_D` **Estimated level:** Challenge
Official task: Open the problem on VJudge
Revision focus: Two stacks; nested-area accounting. Calculate water-filled areas from a cross-section drawn with slash characters. **Suggested approach:** Use a position stack for matching downhill/uphill slopes and a secondary stack for nested pooled areas. **Practice check:** How does merging nested water areas affect the reported sequence of pooled areas? **Complexity guide:** O(n) scan with stacks; O(n) storage

Optional G – Insert or Erase

Judge ID: `AtCoder-abc344_e` **Estimated level:** Challenge
Official task: Open the problem on VJudge
Revision focus: Linked list + HashMap fast lookup. Insert a value after a named existing element, or remove a named element under large query limits. **Suggested approach:** Use HashMap<Integer,Node> plus custom doubly linked nodes for expected O(1) lookup and edits. **Practice check:** Which four links need attention when removing an interior node? What about head or tail? **Complexity guide:** O(n+q) expected with hashing; O(n+q) memory

Optional H – Many Requirements

Judge ID: `AtCoder-abc165_c` **Estimated level:** Challenge
Official task: Open the problem on VJudge
Revision focus: Recursive nondecreasing sequence enumeration. Explore valid nondecreasing integer sequences and maximize a score defined by position relationships. **Suggested approach:** Use DFS to choose each next value not smaller than the previous, then evaluate the complete sequence. **Practice check:** What bounds on the next number preserve the nondecreasing invariant? **Complexity guide:** Exponential combinations in worst case; O(n+q) working state

Optional I – Worms

Judge ID: `CodeForces-474B` **Estimated level:** Easy–Medium
Official task: Open the problem on VJudge
Revision focus: Prefix sums + binary search. Find which pile contains a given numbered worm using cumulative pile sizes. **Suggested approach:** Construct prefix totals and binary-search the first cumulative total not smaller than the query. **Practice check:** Which pile owns a query equal to a cumulative boundary? **Complexity guide:** O(n + q log n) time; O(n) prefix storage

Optional J – Make Takahashi Happy

Judge ID: `AtCoder-abc293_c` **Estimated level:** Challenge
Official task: Open the problem on VJudge
Revision focus: DFS recursion; grid backtracking. Count paths from top-left to bottom-right on a small grid with no repeated visited values. **Suggested approach:** At each cell, recursively try down/right while maintaining a set of values on the current path; undo on return. **Practice check:** What data must be removed from the set when backtracking to another branch? **Complexity guide:** O(C(H+W-2,H-1)*(H+W)) worst; O(H+W) DFS state

Submission and Review Checklist

What students submit

- Join your assigned **VJudge contest** using the link and access code shared on Blackboard; do not substitute links from another version.
- Submit **four accepted Java solutions** for tasks A, B, C, and D before the deadline. E–J are optional.
- Use the exact judge input/output specification. Do not print debug traces, explanatory text, or custom labels to standard output.
- Keep your accepted Java code so the instructor can inspect whether linked lists and recursion were actually implemented.
- A separate ZIP of Java files and a PDF report are **not required by this revised VJudge submission format** unless the instructor explicitly assigns additional written evidence on Blackboard.

Personal midterm revision checks (ungraded)

1. Show a complete pass-by-pass trace of bubble, insertion, and selection sorts using the shared tutorial example.
2. Trace binary search with low/mid/high both when the target exists and when it does not.
3. Explain the differences between array, circular, linked, and priority queues and the cost of common operations.
4. Diagram head/tail/prev/next updates for a singly or doubly linked list.
5. Write the base case, recursive step, and call/return trace of your recursion solution.

Final learning objective

Use the appropriate algorithm and data representation; justify correctness and complexity; test the boundaries; and obtain accepted Java solutions on the assigned VJudge contest. The theoretical revision and visual traces remain the basis for the midterm.